

Le Workflow Homelab

Un cadre de collaboration entre intelligences artificielles spécialisées, piloté par l'humain, pour créer et faire évoluer les stacks, la configuration et les services d'un Homelab avec rigueur, traçabilité et sécurité.

Coordination multi-agents (A2A)

Validation humaine à chaque étape

Sécurité systématique

Docker Swarm · Terraform · n8n · Home Assistant

Portable & agnostique

1 Contexte & objectifs

Le **workflow Homelab** est la méthode de travail d'une équipe d'**agents d'intelligence artificielle spécialisés** en DevOps Homelab. Chaque demande — créer une stack, faire évoluer une configuration, corriger un paramètre, appliquer un correctif de sécurité — est traitée comme un chantier encadré, où un **Tech Lead** orchestre des spécialistes et où **l'humain décide à chaque jalon**.

Le contexte

Un Homelab mêle des stacks Docker Swarm, de l'infrastructure Terraform / Proxmox, des flux d'automatisation (n8n) et de la domotique (Home Assistant). Ces changements touchent des secrets, des réseaux, du routage Traefik et des déploiements. Le workflow Homelab apporte un cadre commun : des étapes claires, des rôles bien définis et une gouvernance qui garde l'humain aux commandes.

Les objectifs

- **Adapter l'effort au besoin** — léger pour un simple paramètre, complet pour une nouvelle stack.
- **Garantir la traçabilité** — chaque décision est consignée sur l'issue et justifiée.
- **Sécuriser par défaut** — un contrôle sécurité systématique (secrets, exposition, hardening).
- **Garder l'humain décideur** — aucune action à impact sans sa validation explicite.
- **Apprendre dans le temps** — les corrections deviennent des règles durables.

Principe fondateur : le workflow s'adapte au travail, et non l'inverse. Un simple changement de variable reçoit un traitement minimal ; une nouvelle stack ou un correctif de sécurité reçoit un traitement complet et renforcé. L'effort est calibré selon l'intention, l'état existant de la stack, la complexité et le risque.

2 Fonctionnement global : cinq phases

Un chantier avance en **cinq phases successives**. Entre chaque phase, un **point de contrôle** (le « gate ») laisse l'humain approuver avant de continuer. Plus on avance, plus la validation est exigeante.

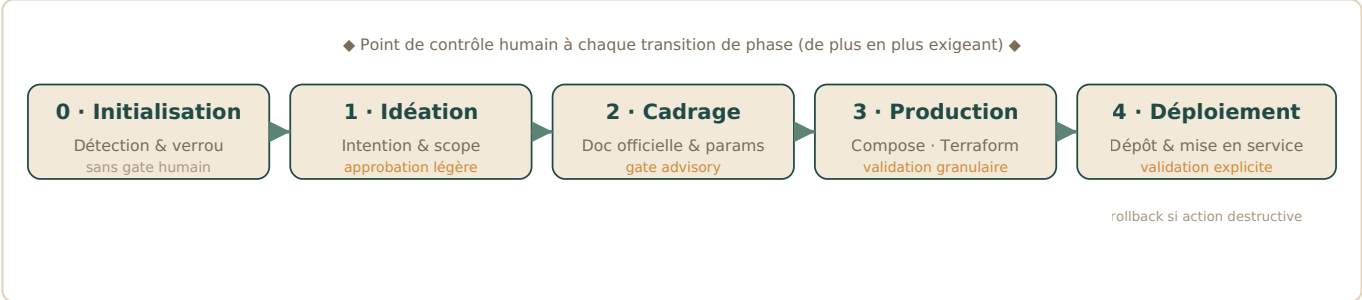


Figure 1 — Les cinq phases du workflow Homelab et l'exigence croissante de validation humaine.

Phase	Ce qui s'y passe	Contrôle humain
0 - Initialisation	Bootstrap déterministe : détection de la stack, lecture du verrou de concurrence, pré-vérification des pré-requis de déploiement, audit des labels.	Automatique aucun gate
1 - Idéation	Capture de l'intention, arbitrage de faisabilité, détection & confirmation du scope, présélection du type d'authentification.	Approbation de l'intention et du périmètre léger
2 - Cadrage et Paramètres	Règle absolue n8n, triage de domaine, cadrage sur la documentation officielle, arbitrage Swarm / Proxmox, collecte des paramètres requis.	Gate advisory avant Production
3 - Production et Contrôle	Mode autonomie, configuration Terraform, création du docker-compose, assurance qualité, branches n8n / Home Assistant, contrôle qualité central.	Validation choix par choix
4 - Validation et Déploiement	Pré-requis de déploiement, revue & notification, validation humaine granulaire, dépôt des fichiers, flux Kestra, clôture.	Validation explicite + rollback

Les libellés historiques (Cadrage et Paramètres, Production et Contrôle, Validation et Déploiement) restent valides comme alias des phases Inception / Construction / Operation.

3 Les méthodes du workflow

3.1 — Le cycle d'une étape (6 temps)

Chaque étape (ou « stage ») suit le même rythme. Ce canevas garantit qu'aucune production (docker-compose, configuration Terraform, flux) n'échappe au contrôle qualité, sécurité et humain.

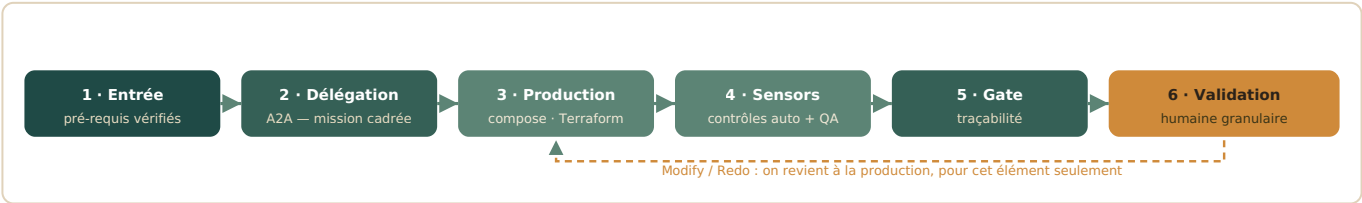


Figure 2 — Le cycle en 6 temps d'une étape, de l'entrée à la validation humaine.

3.2 — La décision humaine : Keep / Modify / Redo

À chaque point de validation, le Tech Lead présente **chaque choix séparément** (le choix, sa justification, une alternative). L'humain tranche **élément par élément** :

Keep Le choix est validé, on avance.

Modify L'humain reformule, on réajuste cet élément.

Redo Le choix est rejeté, on propose une alternative.

Aucun élément non validé n'avance. Jamais d'approbation globale « tout ou rien » — même en mode autonome.

3.3 — Calibrer l'effort : scopes & axes

Le principe « le workflow s'adapte au travail » est outillé par des **scopes** (le type de chantier, qui détermine les étapes exécutées) et par **deux réglages indépendants** : la **profondeur** du détail produit (Depth) et le **niveau de vérification QA**.

7 scopes types

stack-update (défaut) · new-stack · config-change · security-patch · infra-terraform · n8n · home-assistant

Le scope est auto-déTECTÉ par mots-clés puis **confirmé explicitement** avant de démarrer — jamais de démarrage silencieux. Le doute ne bascule jamais vers l'allégé ; priorité au niveau le plus élevé.

2 axes de réglage

- Depth** : minimal · standard · comprehensive (combien on écrit)
- Vérification** : advisory · standard · renforcé (à quel point on vérifie)

Sur **security-patch** et **new-stack**, le plancher (Depth ≥ standard, vérification renforcée) ne peut **jamais** être abaissé sans validation humaine tracée.

Scope	Intention type	Depth	Vérification
stack-update défaut	Modification d'une stack existante	standard	standard
new-stack	Création complète d'une nouvelle stack	comprehensive	renforcé 🔒
config-change	Variable existante sans impact sécurité (≈ allégé)	minimal	advisory
security-patch	Tout impact sécurité (auth, réseau, secrets, hardening, Traefik)	comprehensive	renforcé 🔒
infra-terraform	Infrastructure Terraform / Proxmox	standard	standard
n8n	Toute demande n8n — branche autonome	standard	standard
home-assistant	Toute demande Home Assistant — branche autonome	standard	standard

Chaque scope est détaillé — intention, parcours, axes, garde-fous et **diagramme d'état avec interactions A2A & Humain** — dans l'**Annexe (section 8)**.

3.4 — La mémoire de règles (apprentissage)

Le workflow **apprend des corrections humaines** : quand une personne corrige ou rejette un choix (ou qu'un défaut de convention revient — secret sans `_FILE`, section `deploy` manquante, réseau Traefik oublié), cela devient un **candidat-règle**. Après confirmation explicite et contrôle de conflit, la règle est écrite et **s'appliquera au prochain chantier** — jamais rétroactivement. La mémoire est organisée en **quatre couches** de priorité décroissante.

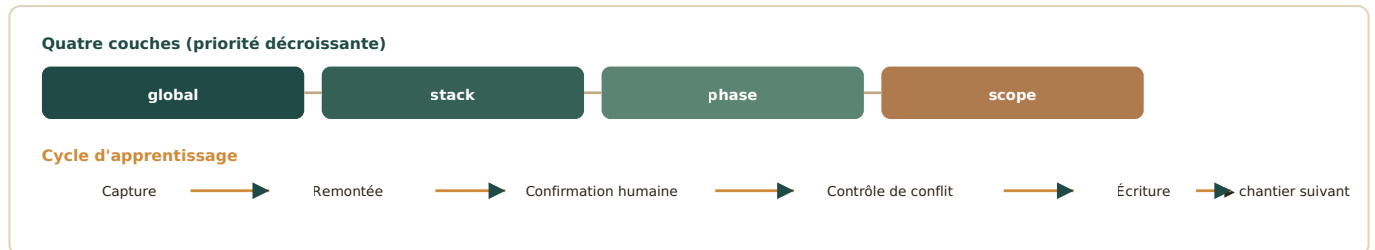


Figure 3 — Couches de la mémoire de règles (global > stack > phase > scope) et cycle d'apprentissage.

Un candidat qui affaiblirait un invariant ou un garde-fou est rejeté d'office. La promotion vers **global** passe par le contrôle sécurité de l'Architecte de sécurité Homelab.

3.5 — Garde-fous & contrôles automatiques

Invariants non contournables

- Règle absolue n8n (délégation immédiate, pas même l'analyse)
- Sélection auto du type d'authentification (`oidc` → `forwardauth` → `local`)
- Validation humaine granulaire ; aucune action à impact sans accord explicite
- Terraform ne déploie jamais ; aucun secret en clair ; jamais `${SNI}` en Terraform livré
- Un seul traitement en cours par stack (verrou `active_step`)
- Piste d'audit sur l'issue ; décision structurante tracée en ADR

Gates & sensors

Des **verification gates** (traçabilité, aux frontières de phases) et **six sensors** déterministes (validité YAML, section `deploy` Swarm, secret en clair, absence de `${SNI}`, cohérence Traefik, existence des secrets Vault) assistent la décision.

Advisory par défaut : ils ne bloquent pas et ne remplacent ni la validation humaine ni le contrôle sécurité.
Exception : `plaintext-secret` et `terraform-no-sni` sont **bloquants** sur `security-patch` / `new-stack`. Jamais de lecture de valeur de secret : emplacement et existence seulement.

4 Les agents & les compétences mobilisés

Le workflow réunit une **équipe DevOps Homelab**, coordonnée par un chef d'orchestre. Le **Tech Lead ne produit pas les livrables** : il analyse, découpe, délègue, contrôle et fait valider. Les spécialistes produisent dans leur domaine. Le Tech Lead ne réalise lui-même que les domaines sans agent dédié (Ansible, logs, Kestra), en le signalant à l'humain.

Agent (fonction)	À quoi il sert
Tech Lead Homelab coordinateur + contrôle qualité central	Orchestre le workflow : analyse la demande, applique la règle de documentation officielle, pose le verrou par stack, délègue, sollicite le contrôle sécurité, demande les validations humaines, met les livrables à disposition. Aucune issue ne part en revue sans son contrôle.
Spécialiste Docker	Crée / modifie les docker-compose optimisés Swarm (relevé fin de la configuration : variables, conventions <code>_FILE</code> , volumes, healthcheck, hardening). Conserve les commentaires des gabarits.
Spécialiste Terraform	Écrit / modifie les <code>.tf</code> / <code>.tfvars</code> . N'exécute jamais <code>init</code> / <code>apply</code> / <code>destroy</code> ; jamais <code>\${SNI}</code> dans un livrable.
Analyste QA	Vérifie les livrables (YAML, compatibilité Swarm, hardening, cohérence Traefik, config Terraform) et renvoie les défauts à leur créateur sans corriger. Porte le contrôle sécurité technique (revue adversariale) et des manifestes de sensors.
Architecte de sécurité Homelab	Jugement sécurité de posture : hardening et sécurité de base des stacks (secrets, exposition, permissions, durcissement, Traefik). Contrôleur de la couche global de la mémoire de règles. Périmètre limité à la sécurité de base d'un homelab.
Expert n8n	Toute tâche n8n via MCP — branche autonome déclenchée par la règle absolue n8n. Applique après feu vert du Tech Lead et validation humaine explicite.
Expert Home Assistant	Pilote Home Assistant via MCP officiel. Séquence obligatoire : proposition → vérification Tech Lead → validation humaine explicite → modification réelle.
Agent de notifications	Notifie (ntfy) la fin ou la mise en revue, déclenché uniquement par le Tech Lead ; les spécialistes ne l'appellent jamais directement.

Les compétences (« skills ») mobilisées

- **docker-composer** — création de docker-compose optimisés Swarm.
- **configuration-applications** — configuration Terraform des applications.
- **dockerfile-validator** — vérification de la forme des images / Dockerfile.

- **homelab-vault-access** — accès Vault en présence seulement, jamais la valeur.
- **traefik-manager-read** — lecture de la cohérence du routage Traefik.

Le workflow Homelab (triptyque `conductor.md` / `stages/` / `protocols/`) est le workflow : il fixe la gouvernance et l'ordre d'exécution entre agents, et ne remplace pas ces compétences — il en orchestre l'usage.

5 L'A2A : comment les agents collaborent

« A2A » désigne la collaboration **d'agent à agent**. Dans le workflow, elle repose sur un principe simple : **tout passe par l'issue** (le fil du chantier). Le Tech Lead délègue en publiant un commentaire qui **mentionne** l'agent voulu — via un lien mention://agent/<uuid>, l'UUID étant toujours résolu, jamais deviné — et lui confie une **mission cadrée** (objectif, périmètre, critères d'acceptation). L'agent produit, puis mentionne en retour le Tech Lead pour contrôle.

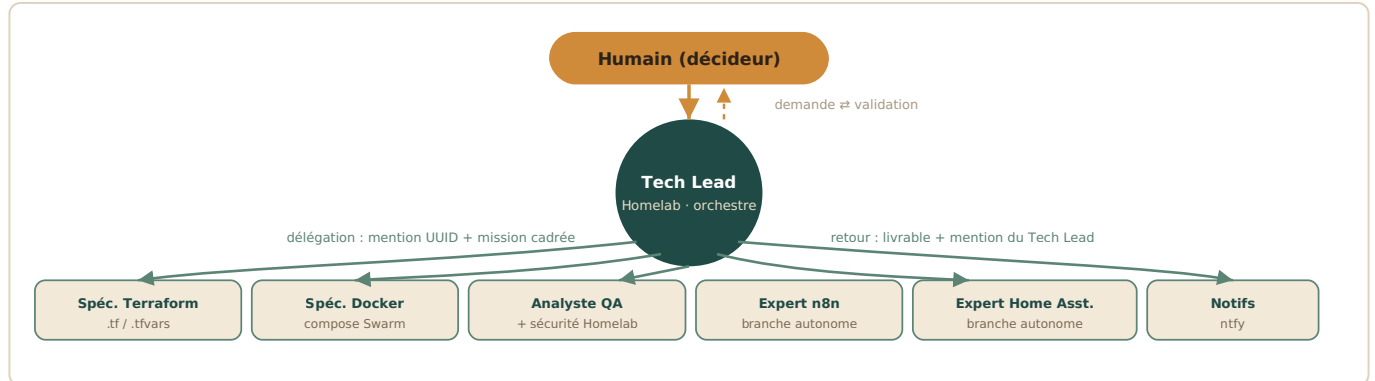


Figure 4 — Le Tech Lead délègue en « étoile » (hub-and-spoke) ; chaque livraison revient pour contrôle qualité central avant validation humaine.

5.1 — États & interactions : A2A, Agent ↔ Humain, gates

Au-delà de la topologie « en étoile », le chantier est une **séquence d'interactions** rythmée par les **gates** (points de contrôle) et par l'**état de l'issue** qui évolue à chaque jalon. Le diagramme se lit de haut en bas : chaque **flèche horizontale** est un message qui relie deux acteurs — en **vert l'A2A** (entre agents), en **ocre l'Agent ↔ Humain** — chaque **losange** un gate, et la colonne de droite donne l'**état de l'issue**. Les acteurs qui **construisent** (Spécialiste Docker, Spécialiste Terraform) et celui qui **vérifie** (Analyste QA) ont chacun leur couloir, et la **notification** finale (Agent de notifications · ntfy) est explicite.

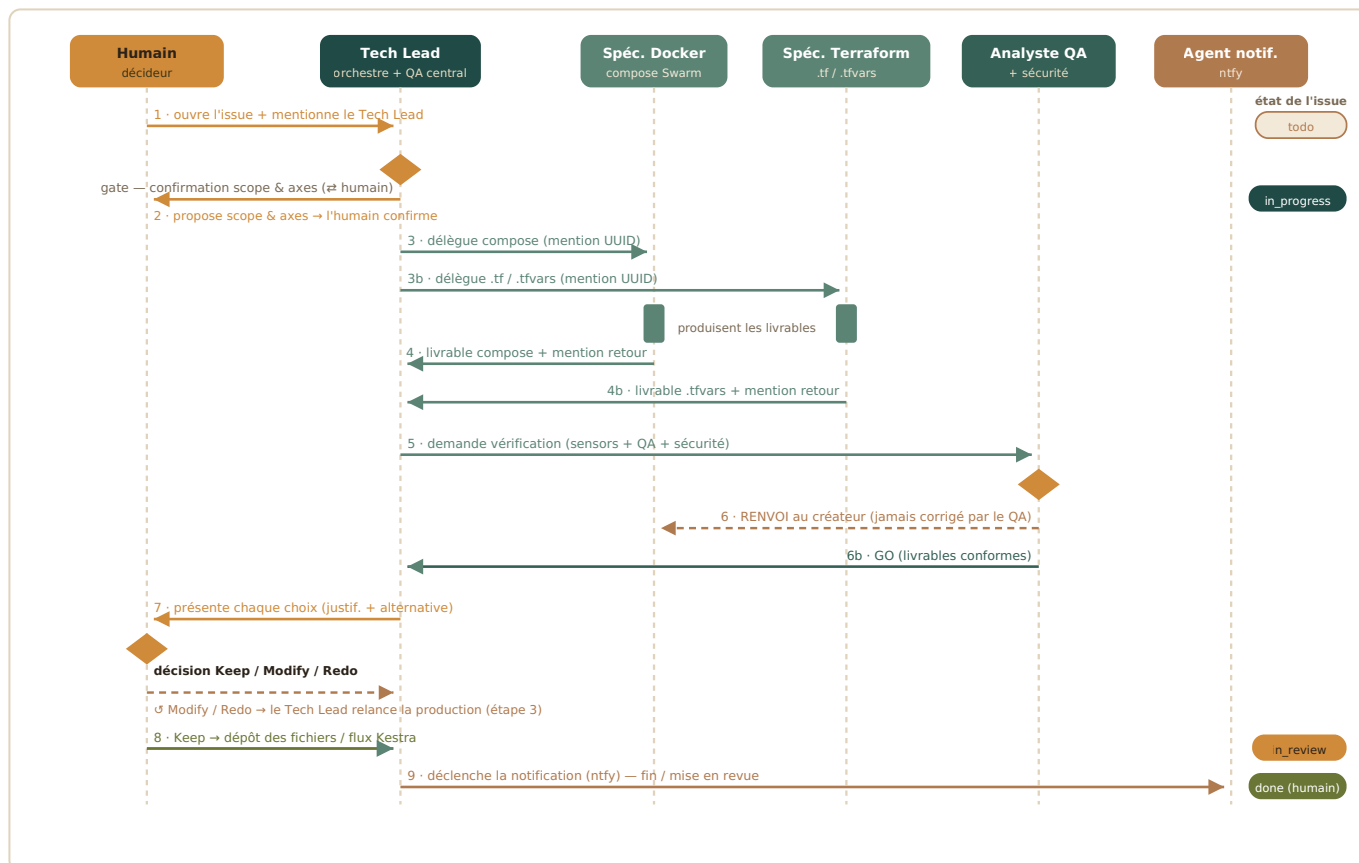


Figure 5 — Diagramme d'états & interactions. Couloirs : Humain, Tech Lead, les deux **constructeurs** (Spéc. Docker, Spéc. Terraform), l'**Analyste QA** et l'**Agent de notifications**. Flèches horizontales = messages (vert A2A, ocre Agent ↔ Humain) ; losanges = gates ; colonne de droite = état de l'issue. Boucles tracées : RENVOI (QA → créateur) et Modify / Redo (humain → production).

Les règles de la collaboration

- Déclenchement par **mention valide** mention://agent/<uuid> + mission claire ; un nom en texte brut ne déclenche rien.
- Le Tech Lead **contrôle chaque livrable** (GO / RENVOI) avant la validation humaine.
- L'agent sollicité **mentionne en retour** le Tech Lead ; sans lien valide, le compte-rendu est réputé non rendu.
- Un **contrôle sécurité** est sollicité dès qu'une surface de sécurité évolue.
- La **notification** (ntfy) est déclenchée uniquement par le Tech Lead ; les spécialistes ne l'appellent jamais.
- Un seul traitement par stack : les demandes concurrentes sont sérialisées (verrou active_step).

Les branches autonomes

Deux domaines **court-circuitent le flux principal** :

- **n8n** — règle absolue : dès que « n8n » apparaît explicitement, délégation immédiate à l'Expert n8n, pas même l'analyse.
- **Home Assistant** — branche autonome vers l'Expert HA, avec double validation avant toute action réelle.

Une demande ambiguë hors stack passe d'abord par le **triage de domaine** (recherche de correspondance MCP, existence seulement), jamais tranchée au jugé.

Toujours sous contrôle humain. Quelle que soit la façon de collaborer, dès qu'un blocage survient — information manquante, contradiction de règles, mention en échec — l'agent concerné **mentionne explicitement l'humain**, passe l'issue en blocked et met le chantier en pause : il n'avance jamais en devinant. La décision finale appartient toujours à la personne.

6 Utiliser le workflow avec Multica

Sur Multica, le workflow s'exécute **autour de l'issue** : chaque demande ouvre une issue, et **tout le travail — analyse, délégation, décisions, validations — vit en commentaires** sur cette issue, qui tient lieu de piste d'audit. L'humain déclenche, suit et valide depuis ce même fil.

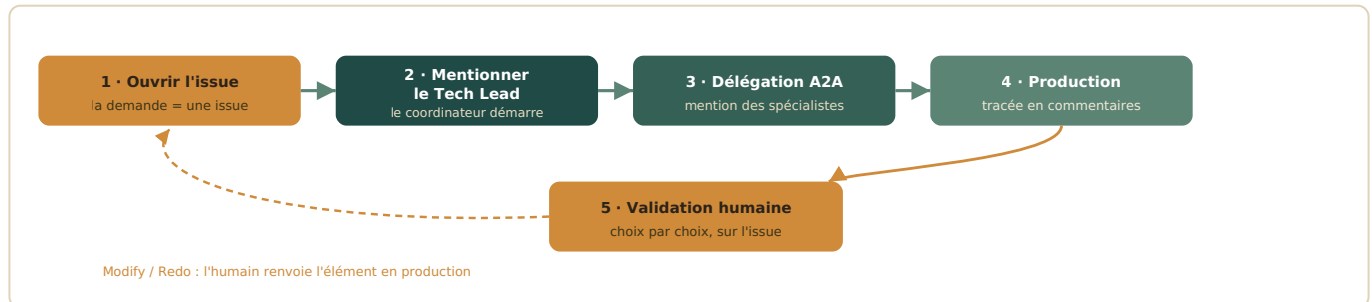


Figure 6 — Le parcours d'une demande sur Multica, de l'ouverture de l'issue à la validation humaine.

Les mécanismes Multica mobilisés

- **L'issue comme pivot** — statut, priorité, assignation et historique au même endroit ; labels Homelab (et Docker Swarm pour les livrables compose).
- **La mention A2A** — déléguer, c'est mentionner l'agent voulu avec une mission cadrée ; l'UUID est résolu automatiquement, jamais deviné.
- **Le verrou par stack** — une clé de metadata `active_step` garantit un seul traitement en cours par stack.
- **La piste d'audit** — chaque étape est un commentaire ; l'historique n'est jamais écrasé.
- **Les pièces jointes** — les livrables (compose, Terraform, rapports) sont attachés à l'issue.

Mettre le workflow en service

- **Agents** — créer l'équipe (Tech Lead + spécialistes), chacun porté par sa définition.
- **Compétences** — les skills (docker-compose, homelab-vault-access, ...) sont d'abord **importées** dans l'espace de travail, puis **assignées** aux agents : présente dans le dépôt, une compétence n'est pas active tant qu'elle n'est pas importée.
- **Démarrage** — ouvrir l'issue, décrire le besoin, mentionner le Tech Lead : le workflow se déroule et sollicite l'humain à chaque jalon.

L'humain reste aux commandes depuis l'issue. Rien n'avance sans sa validation, aucune action à impact (dépôt de fichiers, flux Kestra, application n8n / Home Assistant) n'est engagée sans accord explicite, et tout blocage le mentionne directement pour qu'il tranche.

7 Un workflow agnostique : au-delà de Multica

Le workflow n'est **pas lié à un moteur d'exécution particulier**. Il est entièrement décrit en **documents Markdown lisibles** — les instructions du coordinateur (`conductor.md`), les fiches d'étape, les protocoles, la mémoire de règles, les scopes et les contrôles. Cette **forme déclarative** est sa force : **la méthode est le contenu, pas l'outil** qui l'exécute. Il est aussi **agnostique de la cible** : stacks Docker Swarm ou Proxmox, configuration Terraform, n8n, Home Assistant.

Deux niveaux d'agnosticité

- **Agnostique de cible** — même méthode pour Swarm, Proxmox, Terraform, n8n ou Home Assistant ; le domaine est aiguillé par le scope et le triage, jamais imposé.
- **Agnostique d'outillage** — la description est volontairement séparée de tout moteur technique propriétaire : on adopte **la forme, pas la machinerie**. Les manifestes de gates / sensors sont déclaratifs, non exécutables à ce stade.

Ce qu'un harnais doit fournir

- Lire des **instructions en Markdown** et les suivre.
- Faire tourner des **agents spécialisés** et les **faire dialoguer** (A2A).
- Un **fil partagé** pour tracer les décisions et solliciter l'humain.

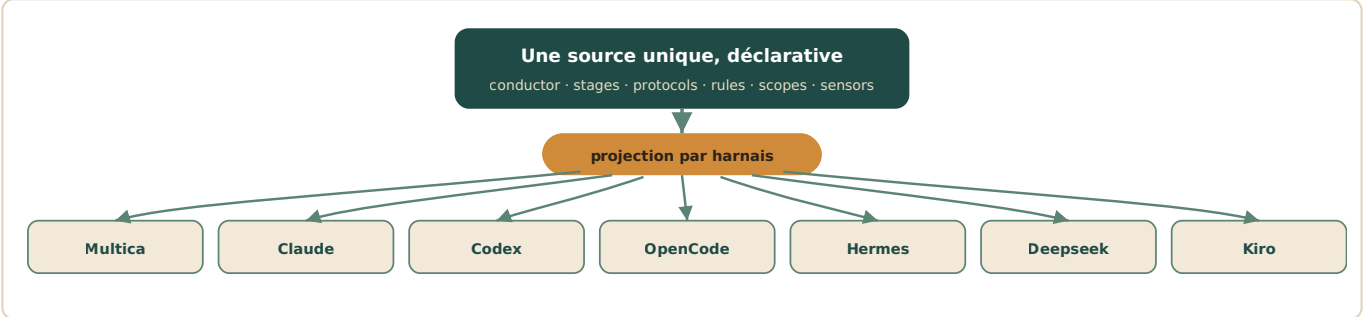


Figure 7 — La même source alimente différents harnais ; seule la couche d'adaptation change, jamais la méthode.

Comment l'utiliser avec un autre harnais

Le passage d'un harnais à l'autre ne touche **ni les phases, ni les étapes, ni les garde-fous** : seule change la **fine couche d'adaptation** qui relie la méthode aux spécificités de l'outil (format d'instruction d'agent, mécanisme de messages, façon de solliciter l'humain).

Harnais	Ce qui change (couche d'adaptation)	Ce qui ne change jamais
Multica	Issue + commentaires, mentions A2A, verrou <code>active_step</code> , pièces jointes — l'exécution native décrite en section 6.	Les 5 phases, le cycle d'étape, la décision Keep / Modify / Redo, les scopes & axes, la mémoire de règles, la règle absolue n8n, le contrôle sécurité et tous les invariants (Terraform ne déploie jamais, aucun secret, jamais <code>\$_SNI</code> , un seul traitement par stack).
Claude	Fichiers d'instructions et de sous-agents au format attendu par l'outil ; options spécifiques projetées depuis la source.	
Codex	Instructions projetées dans sa configuration d'agent ; messages d'étape adaptés à sa surface.	
OpenCode	Définitions d'agents et déclenchements adaptés à son modèle d'exécution.	
Hermes	Même source, projetée dans son mécanisme d'orchestration et de messages.	
Deepseek Harness	Instructions et définitions d'agents projetées dans son format ; messages d'étape adaptés à sa surface d'exécution.	
Kiro		

Agents et instructions portés dans sa configuration native ; déclenchements et sollicitations humaines adaptés à son moteur.

Une méthode, plusieurs moteurs. Parce que la rigueur vit dans les documents — et non dans un outil — le workflow Homelab garde **le même comportement, les mêmes garde-fous et la même qualité** quel que soit le harnais qui l'exécute. On change de moteur sans réécrire la méthode.

8 Annexe — Les scopes en détail

Un **scope** est le **type de chantier** : il fixe le parcours d'étapes déterministe et porte des **valeurs par défaut** pour les deux axes — **Depth** (combien on écrit) et **Vérification** (à quel point on vérifie). Chaque scope est **auto-déTECTÉ par mots-clés** puis **confirmé explicitement** avant de démarrer. Cette annexe détaille, pour chacun des sept scopes, son intention, ses axes par défaut, ses garde-fous propres et un **diagramme d'état** montrant son **flux spécifique**, avec les **interactions A2A** (entre agents, en **vert**) et **Humain** (en **ocre**) mises en avant.

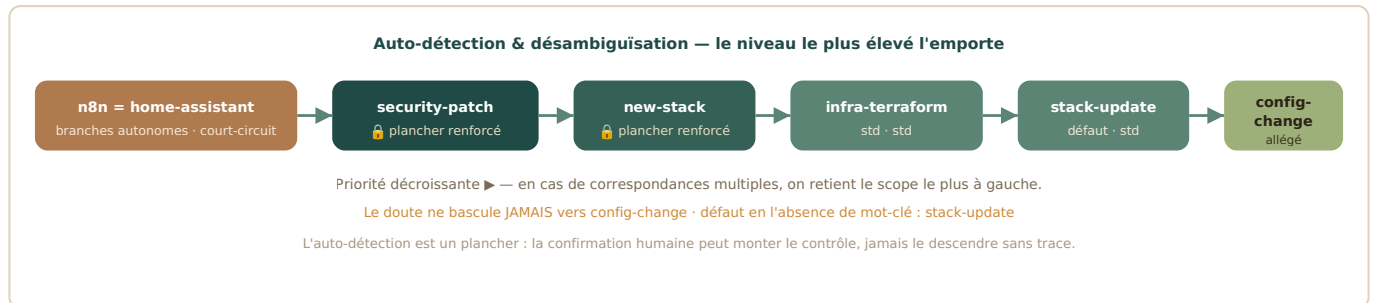
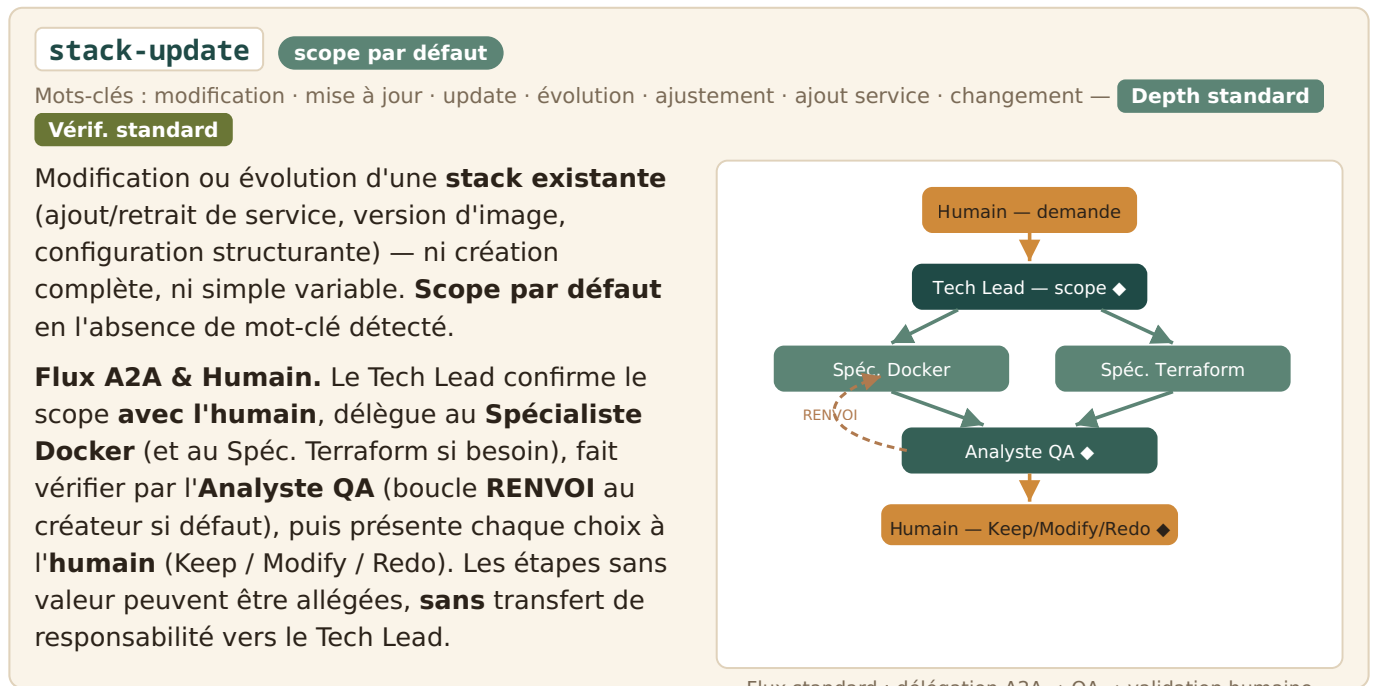


Figure 8 — Ordre de priorité de désambiguïsation des scopes (du plus fort au plus faible).

Légende des diagrammes d'état par scope : — interaction A2A (agent → agent) · — interaction Agent ↔ Humain · - - - boucle de retour (RENOI / Modify / escalade) · ◆ gate / validation.



new-stack

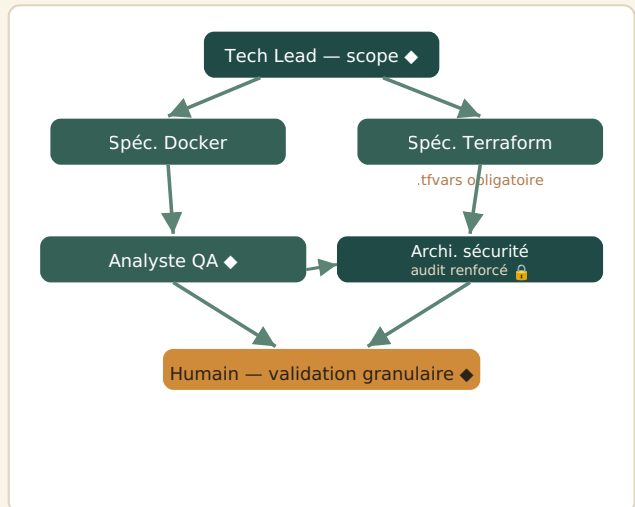
plancher 🔒

Mots-clés : création · nouvelle stack · nouveau service · déployer · installer — **Depth comprehensive**

Vérif. renforcé 🔒

Création complète d'une **nouvelle stack** (docker-compose + Terraform). **Cas le plus exigeant** : toutes les étapes, arbitrage Swarm/Proxmox, collecte exhaustive, sélection auto de l'authentification.

Flux A2A & Humain. Après confirmation du scope **avec l'humain**, le Tech Lead délègue **en parallèle** au Spéc. Docker et au Spéc. Terraform (livrable `.tfvars` **obligatoire**), fait vérifier par l'**Analyste QA** et l'**Architecte de sécurité** (audit renforcé), puis chaque choix passe la **validation humaine**. Plancher non abaissable ; sensors sécurité bloquants.



Chaîne complète + audit sécurité, puis validation humaine

config-change

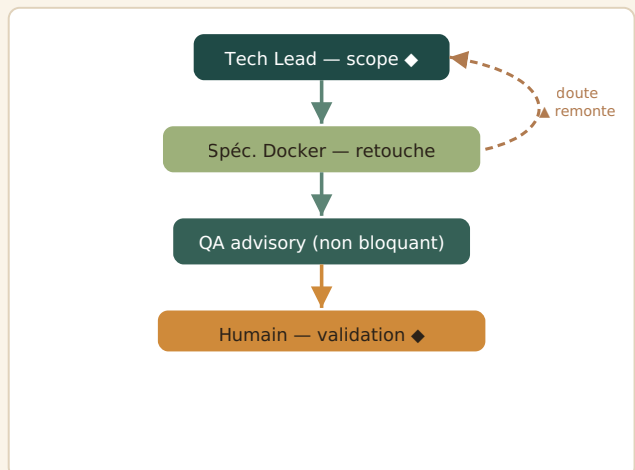
allégé

Mots-clés : variable · valeur · paramètre existant · ajuster valeur · retouche — **Depth minimal** **Vérif. advisory**

Modification d'une **variable existante, sans aucun impact sécurité**. Héritier du « **traitement allégé** » : parcours resserré, moins de contrôles — mais **jamais** de levée de la validation humaine avant action à impact.

Flux A2A & Humain. Chaîne courte : Tech Lead → **Spéc. Docker** (retouche ciblée) → QA **advisory** (signale sans bloquer) → **validation humaine**.

Garde-fou de bascule : au moindre doute sécurité (ou déclencheur d'un scope plus élevé), le travail **remonte** — le doute ne bascule **jamais** vers config-change.



Chaîne courte ; remontée au moindre doute sécurité

security-patch

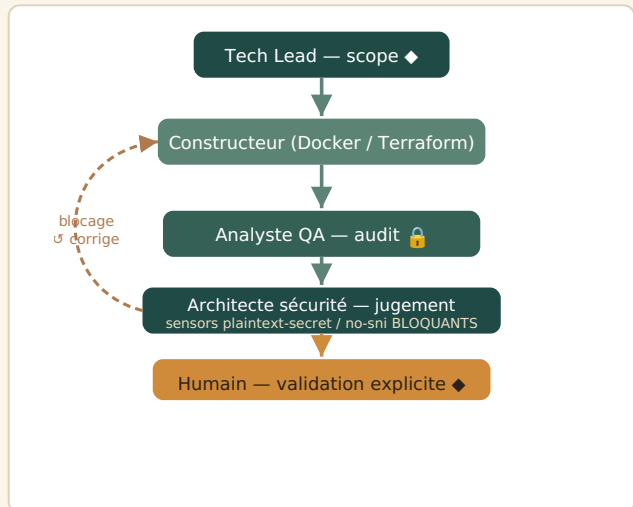
plancher 🔒 · priorité

Mots-clés : sécurité · hardening · secret · auth · exposition · permissions · Traefik · CVE · faille — **Depth comprehensive**

Vérif. renforcé 🔒

Toute modification à impact sécurité (auth, réseau, exposition, secrets, hardening, Traefik). Traçabilité renforcée, **analyse d'impact / non-régression** avant recommandation.

Flux A2A & Humain. Le Tech Lead délègue au constructeur concerné (Docker / Terraform), puis l'**Analyste QA** mène un **audit renforcé** et l'**Architecte de sécurité** rend son **jugement de posture** (A2A). Les sensors plaintext-secret / terraform-no-sni sont **bloquants** (boucle de correction). Enfin, **validation humaine** explicite ; tout abaissement de contrôle exige un accord humain tracé.



Audit QA + sécurité (sensors bloquants) → validation humaine

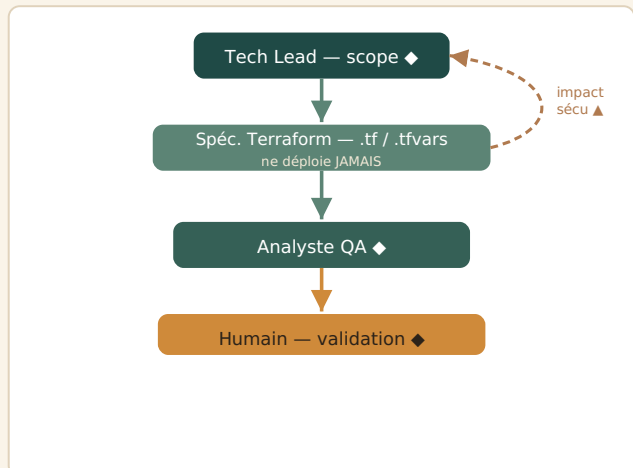
infra-terraform

Mots-clés : terraform · proxmox · infrastructure · tfvars · provider · VM · LXC · réseau infra — **Depth standard**

Vérif. standard

Travaux d'infrastructure centrés **Terraform / Proxmox** (.tfvars, VM / LXC, arbitrage Proxmox vs Swarm). Accent sur le **Spécialiste Terraform**, qui prépare uniquement les fichiers.

Flux A2A & Humain. Tech Lead → **Spécialiste Terraform** (A2A) → **Analyste QA** (vérification config) → **validation humaine**. **Terraform ne déploie JAMAIS** (init/apply/destroy interdits). **Escalade** : si le travail touche auth / exposition / secrets / réseau de façon structurante, il **remonte vers security-patch** ; le doute ne descend jamais.



Spéc. Terraform → QA → humain ; escalade ▲ security-patch

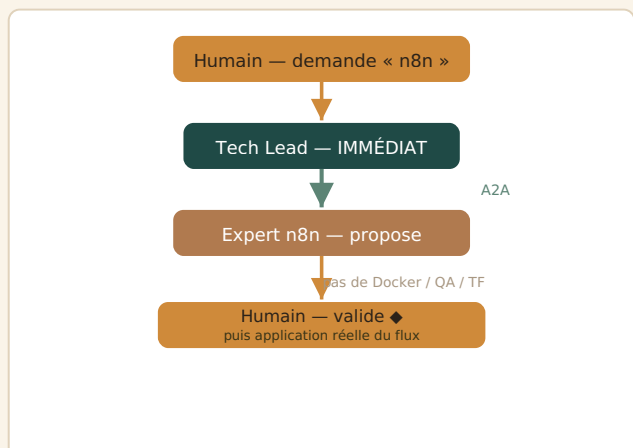
n8n

branche autonome · priorité

Mots-clés : n8n — **Depth standard** **Vérif. standard**

Toute demande **n8n**. **Règle absolue** non désactivable : dès que « n8n » apparaît, le Tech Lead **délègue IMMÉDIATEMENT** à l'**Expert n8n** et **arrête** le flux stack. **Aucune exception, pas même l'analyse.**

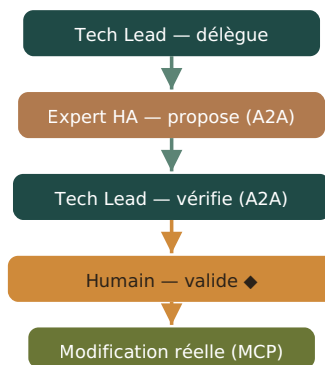
Flux A2A & Humain. Branche **autonome** : ne passe **pas** par Docker / QA / Terraform. L'Expert n8n propose (A2A), le Tech Lead donne son **feu vert**, puis l'**humain valide** explicitement avant toute application réelle du flux. L'Expert mentionne le Tech Lead en fin de travail.



Court-circuit : TL → Expert n8n (A2A) → validation humaine

Toute demande **Home Assistant** (entités, scènes, automatisations, scripts) via le serveur MCP officiel. **Branche autonome** : ne passe **pas** par Docker / QA / Terraform.

Flux A2A & Humain — double validation. Tech Lead → **Expert HA** qui propose (A2A) → **vérification par le Tech Lead** (A2A) → **validation humaine explicite** → **seulement ensuite** la modification réelle. L'Expert HA mentionne le Tech Lead en fin de travail. Prioritaire (même rang que n8n).



Double validation (Tech Lead + humain) avant action réelle

Les axes se relèvent, jamais s'abaissent sans trace. À l'invocation, à la confirmation de scope ou à un verification gate, un niveau peut être **monté**. Tout abaissement d'un niveau lié à la sécurité exige une **validation humaine explicite tracée** sur l'issue — et sur security-patch / new-stack, le plancher (Depth ≥ standard, vérification ≥ renforcé) ne peut jamais l'être. La **source d'identité** de chaque scope reste son fichier homelab/scopes/<nom>.md ; cette annexe en est une vue lisible.